

Callbacks Made Simple

each example is complete: how to write it, and how to call it

DR

Dinesh Rawat

Instructor · Shorter Loop × Snapied

What we are learning. Some jobs take time, so we finish one before starting the next. Today: callbacks, the mess they can make (callback hell), and three ways to fix that mess. Every example is complete, you see the function AND the line that calls it. Type them all and run them.

1 Callback

A callback is a function you give to another function, to run when the job is done. Look for the **call** line in each: that is where we actually run it.

Example 1: Food

```
function orderFood(callback) {  
  console.log("Ordered food");  
  callback();  
}  
  
orderFood(function() {           // <-- calling it  
  console.log("Eat!");  
});
```

Example 2: Alarm

```
function setAlarm(callback) {  
  console.log("Alarm ringing");  
  callback();  
}  
  
setAlarm(function() {           // <-- calling it  
  console.log("Wake up!");  
});
```

Example 3: Message

```
function sendMsg(callback) {  
  console.log("Message sent");  
  callback();  
}  
  
sendMsg(function() {           // <-- calling it  
  console.log("Got a reply!");  
});
```

Every time: the function does its job, then runs the function you passed in the call. That passed-in function is the callback.

When do we use callbacks?

Any time a job does not finish instantly and we must do something **after** it is done. For example:

- Reading or saving a **file** on the computer

- Talking to a **database** (save a user, fetch a list)
- Calling another **website or API** over the internet
- Waiting for a **timer**, like "run this after 5 seconds"
- Waiting for a **user action**, like a button click

All of these take time. The code cannot just stop and wait, so we hand over a callback: "when you finish, run this."

2 Error-first callbacks

In real Node.js, callbacks follow one rule: the first thing they get is the error. Error first, result second. If something went wrong, you get an error. If all is fine, the error is `null` and you get the result. You always check the error first.

Example 1: Read a file

```
function readFile(name, callback) {
  if (!name) {
    return callback("No file name given");    // error first
  }
  callback(null, "File contents here");      // null = no error, then result
}

readFile("notes.txt", function(error, data) { // <-- calling it
  if (error) {
    console.log("Error:", error);
  } else {
    console.log("Got:", data);
  }
});
```

Example 2: Divide

```
function divide(a, b, callback) {
  if (b === 0) {
    return callback("Cannot divide by zero");
  }
  callback(null, a / b);
}

divide(10, 2, function(error, result) {      // <-- calling it
  if (error) {
    console.log("Error:", error);
  } else {
    console.log("Answer:", result);
  }
});
```

Example 3: Find a user

```
function findUser(id, callback) {
  if (!id) {
    return callback("No id given");
  }
  callback(null, { name: "Himanshu" });
}

findUser(5, function(error, user) {           // <-- calling it
  if (error) {
    console.log("Error:", error);
  } else {
    console.log("User:", user.name);
  }
});
```

Why error first? Because you must deal with a problem before you use the answer. Checking the error first, every single time, is the habit that stops your app from crashing later. This is the pattern you will see in every real Node.js project.

3 Callback hell

When steps must run in order, each callback is nested **inside** the one before. First we define the small steps, then we call them nested. Watch the code lean right.

Example 1: Morning

```
function wake(cb) { console.log("Wake"); cb(); }
function brush(cb) { console.log("Brush"); cb(); }
function bath(cb) { console.log("Bath"); cb(); }

wake(function() {           // <-- calling, nested
  brush(function() {
    bath(function() {
      console.log("Ready!");
    });
  });
});
```

Example 2: Tea

```
function boil(cb) { console.log("Boil"); cb(); }
function addTea(cb) { console.log("Add tea"); cb(); }
function pour(cb) { console.log("Pour"); cb(); }

boil(function() {           // <-- calling, nested
  addTea(function() {
    pour(function() {
      console.log("Tea done!");
    });
  });
});
```

Example 3: Shopping

```
function pick(cb) { console.log("Pick"); cb(); }
function pay(cb) { console.log("Pay"); cb(); }
function ship(cb) { console.log("Ship"); cb(); }

pick(function() { // <-- calling, nested
  pay(function() {
    ship(function() {
      console.log("Delivered!");
    });
  });
});
});
```

Why is the nesting needed?

The nesting is not a mistake, it is forced. Here is why:

- Each step must **wait** for the one before to finish. You cannot pour tea before the water boils.
- The result of one step is **needed** by the next. Step 2 uses what step 1 gave back.
- A callback only runs **when its job is done**, so the only way to run the next step at the right time is to put it **inside** the callback of the step before.
- Do that three or four times and each callback ends up inside the last, which is what makes the pyramid.

See the shape? Each call sits inside the one before. The `});` pile up at the bottom. That mess is callback hell. Now, three ways to fix it.

4 Fix 1: break into named functions

The simplest fix, no new tools. Give each step its own name, and let each one call the next by name. No nesting. One clean call starts it.

Example 1: Morning

```
function wake() { console.log("Wake"); brush(); }
function brush() { console.log("Brush"); bath(); }
function bath() { console.log("Bath"); done(); }
function done() { console.log("Ready!"); }

wake(); // <-- one call starts the chain
```

Example 2: Tea

```
function boil() { console.log("Boil"); addTea(); }
function addTea() { console.log("Add tea"); pour(); }
function pour() { console.log("Pour"); ready(); }
function ready() { console.log("Tea done!"); }

boil(); // <-- one call starts the chain
```

Example 3: Shopping

```
function pick() { console.log("Pick"); pay(); }
function pay() { console.log("Pay"); ship(); }
function ship() { console.log("Ship"); done(); }
function done() { console.log("Delivered!"); }

pick(); // <-- one call starts the chain
```

Same steps, same order, but flat. You read it top to bottom, each step has a name, and you start it all with one simple call.

5 Fix 2: Promise

A cleaner way to say "this finishes later." The function returns a Promise. You call it and attach `.then()` to catch the result. No nesting.

Example 1: Cook

```
function cook() {
  return new Promise(function(resolve) {
    resolve("Food ready");
  });
}

cook().then(function(result) { // <-- calling it
  console.log(result);
});
```

Example 2: Download

```
function download() {
  return new Promise(function(resolve) {
    resolve("File downloaded");
  });
}

download().then(function(result) { // <-- calling it
  console.log(result);
});
```

Example 3: Login

```
function login() {
  return new Promise(function(resolve) {
    resolve("Logged in");
  });
}

login().then(function(result) { // <-- calling it
  console.log(result);
});
```

`resolve(...)` means "done, here is the result." You call the function, then `.then(...)` catches that result.

6 Fix 3: async / await

The cleanest way. `await` means "wait here until it is done, then continue." Put `async` on the function to use `await` inside it. Then just call the function.

Example 1: Tea

```
function makeTea() {
  return new Promise(function(resolve) {
    resolve("Tea ready");
  });
}

async function serveTea() {
  const result = await makeTea();
  console.log(result);
}

serveTea(); // <-- calling it
```

Example 2: Data

```
function getData() {
  return new Promise(function(resolve) {
    resolve("Data loaded");
  });
}

async function show() {
  const result = await getData();
  console.log(result);
}

show(); // <-- calling it
```

Example 3: Checkout

```
function pay() {
  return new Promise(function(resolve) {
    resolve("Payment done");
  });
}

async function checkout() {
  const result = await pay();
  console.log(result);
}

checkout(); // <-- calling it
```

No nesting at all. Wait, get the result, move on. This is how modern Node.js is written.

7 Remember this

Callback is a function you run when a job is done.
 Error-first means the callback gets the error first, then the result.
 Callback hell is many callbacks nested inside each other, a mess.
 Named functions flatten the mess, start with one call.
 Promise cleans it with `.then()`.
 async / await is cleanest, reads like normal top-to-bottom steps.

All do the same job: run steps in order. They just get cleaner each time.

8 Best practices

Always check the error first. In an error-first callback, handle the error before you touch the result.

Use `return` when you call back with an error. It stops the function so the success code does not run by mistake.

Give your functions clear names. `handleUser` tells you more than a nameless function buried in nesting.

Do not nest too deep. If you see three or four levels, switch to named functions, Promises, or `async/await`.

Prefer `async/await` for new code. It is the cleanest and easiest to read. Use Promises when a library gives you one.

One callback, called once. Do not call the same callback twice, and do not forget to call it at all.

9 Homework

Type every example and run it. Do not copy paste.

Change the words. Make your own, like `orderTea` or `playSong`.

Say it simply: what is callback hell, and how do named functions, Promises, and `async/await` each fix it?